

Mia Letendre 40274681

Dr. Tiberiu Popa

COMP 438

Project Report

Procedural Vine/Ivy Generation

As an artist and a computer scientist, I always want to make my life easier. It's the reason I and many others code, as it allows us to do things that we want/need to do with even slightly more ease. That is reason that this project is about procedural vine generation. It allows me to make any future art projects simple, with the ability to grow any vine that I would need in seconds. I created this project using Lindenmayer systems, a node based hierarchical tree data structure, Catmull Rom splines, and python implementation for Blender so all the previously mentioned to be made easily in a free 3D modeling software. To understand how all of these were implemented, we must first look at the node class.

While much of the node implementation is standard ie getters, setters, children, and parent, there are some unique features for this specific project. First, since each node is going to be related to a point on the vine, thus it needs to track the position, direction, rotation, radius, and depth to make sure the whole vine is able to track what is happening at each of the points. Next, and most importantly, there needed to be a way to turn those nodes into a trackable 2D array, where each row was a different branch. To turn the node tree into said array, a recursive harvest function was made. It starts by tacking how far in the depth is in relation to the root, then adds the position to the current branch. If the node has no kids, it returns with the information about

the total branches, otherwise, the first child continues the current branch and if there are other children, they all start their own. This makes tracking branches simple for splining later on.

The next two implemented classes were a turtle class and the vineGen class. The Turtle class processes the string provided to it and makes the correct rotations and steps, the way the turtle knows how much to turn and which node is the root is by what was passed to it. The vineGen class is where most of the project is based, it turns the rules, axioms, iterations and turns it into the L string that will be processed, generates the vine from the L string and an angle, and then splines it to make it smooth. When passing the rules as an unordered map, the axiom, and the number of wanted iterations, the getLString function takes what was provided and turns it into a long string that will dictate the vines' growth. Then passing that string and the angle to create a turtle that will process the string, harvest it, and return the array of all the branches. Finally, splineIt takes those branches and applies a Catmull Rom spline, which is the most optimal spline for this situation as it has C1 continuity and interpolation. It takes 4 points of each branch at a time and makes 10 new points from where they were using the formula $P(s) = T * M * G$. Finally, you have a completed splined set of branches to be made in Blender.

The next part of the implementation where the main.cpp file which has the pybind module and the method that makes the total vine with the branches. Then in the vineGen_addon.py file, the points are taken and added to a Blender NURBS curve, with some additional calculations related to depth and thickness. Then finally the flowers are added to the end of each branch and the leafs follow. Finally adding this file to Blender allows you to generate vines based on the rules, angle, axiom, iterations, flowers, and leafs that you want.

The only piece of implementation that was not done for this project was the ability to have it establish collision to cling to meshes. This was far too out of scope for the project, even a simple collision detection would have been nearly impossible as this project was already barely finished. It would have required a lot of time figuring out how to change thousands of points in accordance to an object. By no means was this impossible, but not at all something feasible for this project.

Of all the implementation, none of the C++ was too difficult, some of the splining and harvesting was a bit tricky at first, but reasonable as I continued. What was harder and time-consuming then any other part by far was the `vineGen_addon.py` file. The reason for this was because it took a lot to make sure the branches tapered off and started tapered in a normal way, while also having to completely understand Blenders API and bpy from scratch made this a very hard uphill battle. For example, adding the vine texture was not as simple as getting the path and adding it, I had to understand texture nodes and how they relate to the object. Even understanding the structure of how to make this add-on was difficult. Having to make the panel, the properties and the generation all as different classes with more stuff inside each that was more complicated than the last was cumbersome.

Overall this project is a dedication of love. It was something that allowed me to pour my heart into and create an addon that could be used far longer than just this class. It has made me a better developer and hopefully with encourage me to make more addons in the future. All the implementation of the node, turtle and vine generator made this a challenging but enticing

experience. I look forward to the future of creating more for this or creating other similar projects.

Works Cited

- "Bark Willow." Poly Haven, polyhaven.com/a/bark_willow. Accessed 20 Mar. 2026.
- Blender Foundation. Blender. Version 5.0.1, Blender Foundation, www.blender.org. Accessed 10 Mar. 2026.
- Twigg, Christopher. "catmullRom.pdf." Carnegie Mellon University, www.cs.cmu.edu/~fp/courses/graphics/asst5/catmullRom.pdf. Accessed 13 Mar. 2026.
- "Detailed Closeup Single Grape Leaf." Freepik, www.freepik.com/free-psd/detailed-closeup-single-grape-leaf_409868960.htm. Accessed 20 Mar. 2026.
- "Curves and Splines" Computer Graphics (15-462), Spring 2010, Carnegie Mellon University, www.cs.cmu.edu/afs/cs/academic/class/15462-s10/www/lec-slides/lec06.pdf. Accessed 13 Mar. 2026.
- "Pink Flower Petal Illustration Vector Set." Freepik, www.freepik.com/free-vector/pink-flower-petal-illustration-vector-set_16359014.htm. Accessed 20 Mar. 2026.
- Futrick, S. Robert, and Jason Stowe. "LSystems Generator." Cornell University, people.ece.cornell.edu/land/OldStudentProjects/cs490-96to97/STOWE/P3D.html. Accessed 14 Mar. 2026.